Advanced AI Engineering: LLMs, Agents and RAG

Independent study notes for an AI Software Engineering bootcamp. These are original summaries, not official course materials.

1. Large Language Models

A large language model (LLM) predicts and generates token sequences based on learned patterns. Tokens are units of text; the context window limits how much input and output the model can handle at once. Temperature and other sampling settings affect variability, but they do not guarantee correctness.

LLMs can produce plausible but unsupported claims, often called hallucinations. Grounding an answer in trusted source material and checking citations can reduce, but not eliminate, this risk.

2. AI agents and tools

An AI agent uses a model to choose actions toward a goal, often by calling tools, inspecting results, and iterating. Tool access must be constrained by permissions, validation, and logging. A deterministic workflow is often safer than an open-ended agent when the steps are known.

Example: an assistant searches internal documentation, reads a relevant excerpt, and drafts an answer. It should ask for confirmation before taking consequential external actions.

3. Context engineering

Context engineering selects and structures the information supplied to a model: instructions, recent conversation, retrieved documents, tool results, and relevant state. Good context is relevant, current, compact, and clearly labeled by trust level.

Avoid dumping whole repositories or long histories when a small excerpt is enough. Summarize stable decisions and retrieve detailed evidence only when needed. This can lower token usage and reduce distraction.

4. Model Context Protocol (MCP)

Model Context Protocol (MCP) is an open protocol for connecting AI applications to external tools and contextual resources through standardized interfaces. An MCP server can expose tools, resources, and prompts to a compatible client.

MCP does not itself make model answers reliable. The application still needs authentication, authorization, validation, and safe handling of external content.

5. LangChain

LangChain is a framework for assembling LLM applications using components such as models, prompts, document loaders, text splitters, retrievers, and tools. It can support RAG pipelines, although a small application may also implement retrieval with direct SDK calls.

A typical pipeline loads PDF text, splits it into chunks, embeds each chunk, stores vectors, retrieves similar chunks for a question, and passes those chunks to a model for answer generation.

6. Embeddings and vector databases

An embedding is a numerical representation of text that enables similarity comparisons. A vector database such as Pinecone stores embeddings together with IDs and metadata. Queries are embedded using a compatible model and matched against stored vectors.

Metadata can include source_file, page, chunk_index, and document_title. Metadata filtering can narrow retrieval to a particular document. Embedding dimensions must match the index configuration.

7. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) combines external document retrieval with model generation. At indexing time, documents are loaded, split into chunks, embedded, and stored. At question time, the system embeds the query, retrieves relevant chunks, and gives them to the model as evidence.

RAG is useful when information is private, frequently updated, or too extensive for a prompt. Its quality depends on document extraction, chunk boundaries, embeddings, retrieval ranking, and answer instructions. A high similarity score alone does not prove that a chunk answers the question.

8. Chunking and retrieval evaluation

Chunking divides long documents into smaller segments. A starting strategy is 500-800 characters per chunk with 80-120 characters of overlap; adjust based on headings, token limits, and observed retrieval quality. Preserve source and page metadata where possible.

Evaluate retrieval with answerable questions and at least one unanswerable question. For example, the question "What is MCP?" is answerable from this document, while "What is the current weather in Berlin?" is not. A grounded assistant should state that the documents do not contain the answer instead of guessing.

9. Example RAG acceptance criteria

Given two indexed PDFs, the system should retrieve source excerpts for questions about Git, MCP, or RAG; include the source document in its response; and explicitly abstain when evidence is absent. Tests should confirm that documents were actually indexed and that retrieved chunks originate from the selected PDFs.